Scaling MQTT — System Design Perspective

MQTT is designed for **massive numbers of long-lived, lightweight connections**, which makes scaling it fundamentally different from scaling REST APIs.

For a system-design interview, the key question is:

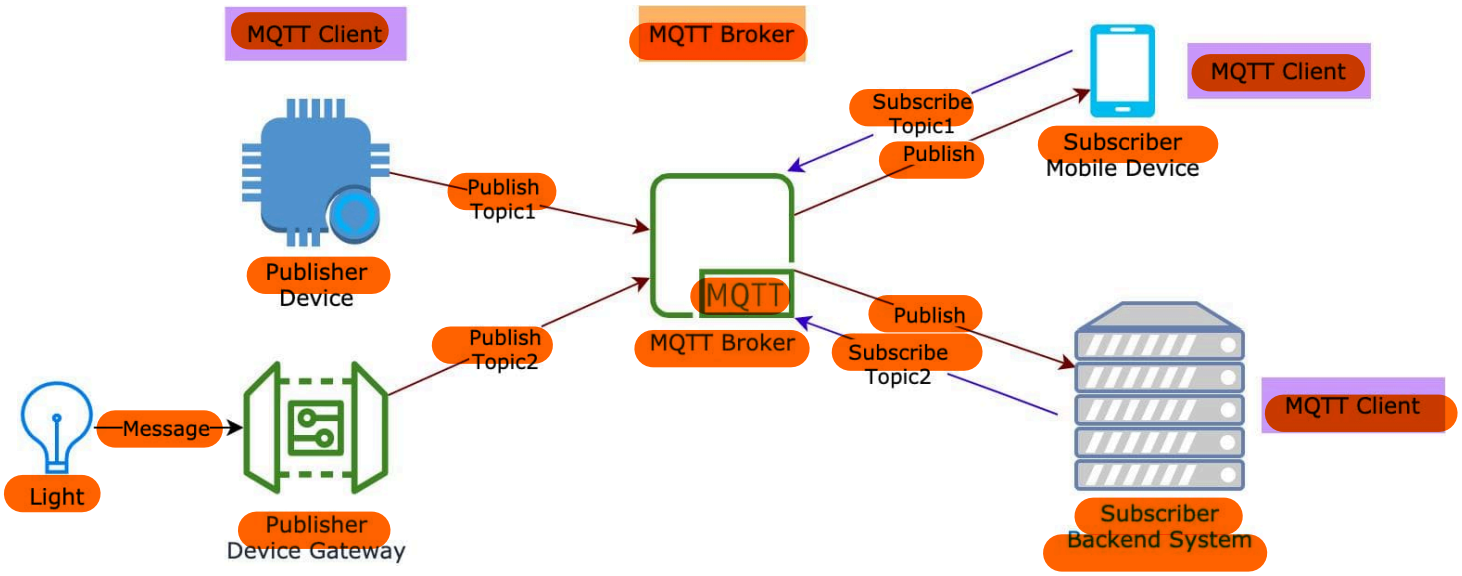
> **How do we support millions of simultaneously connected devices while maintaining low-latency publish/subscribe messaging, reliable delivery, and acceptable broker resource usage?**

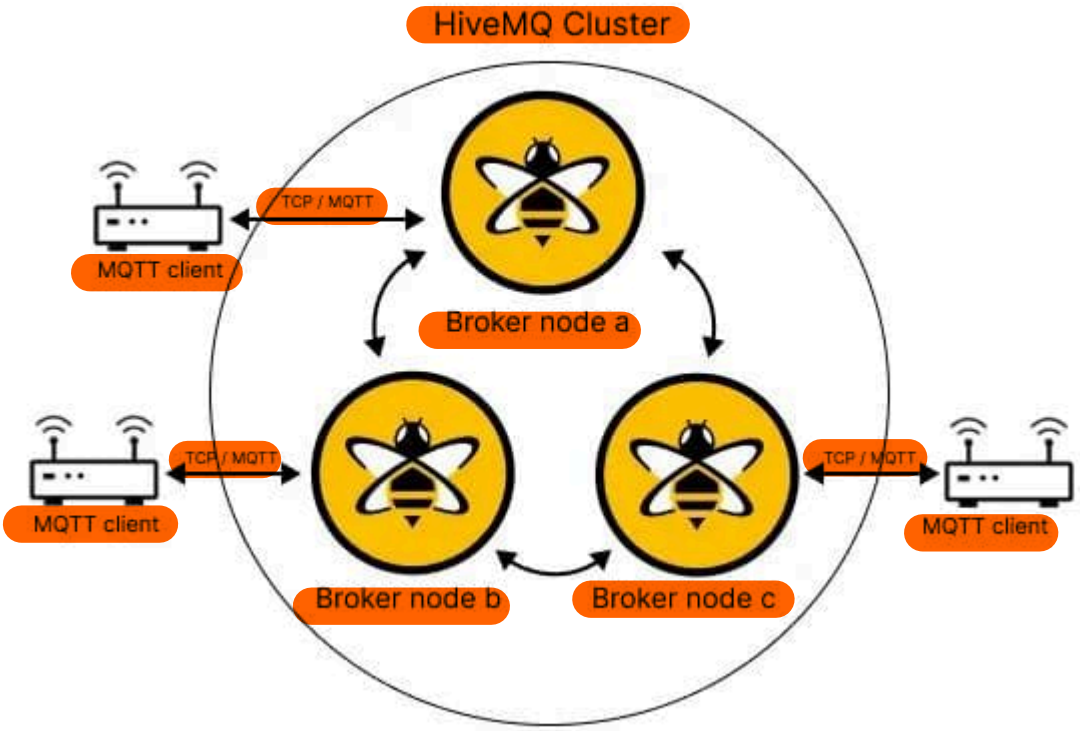
The central idea is:

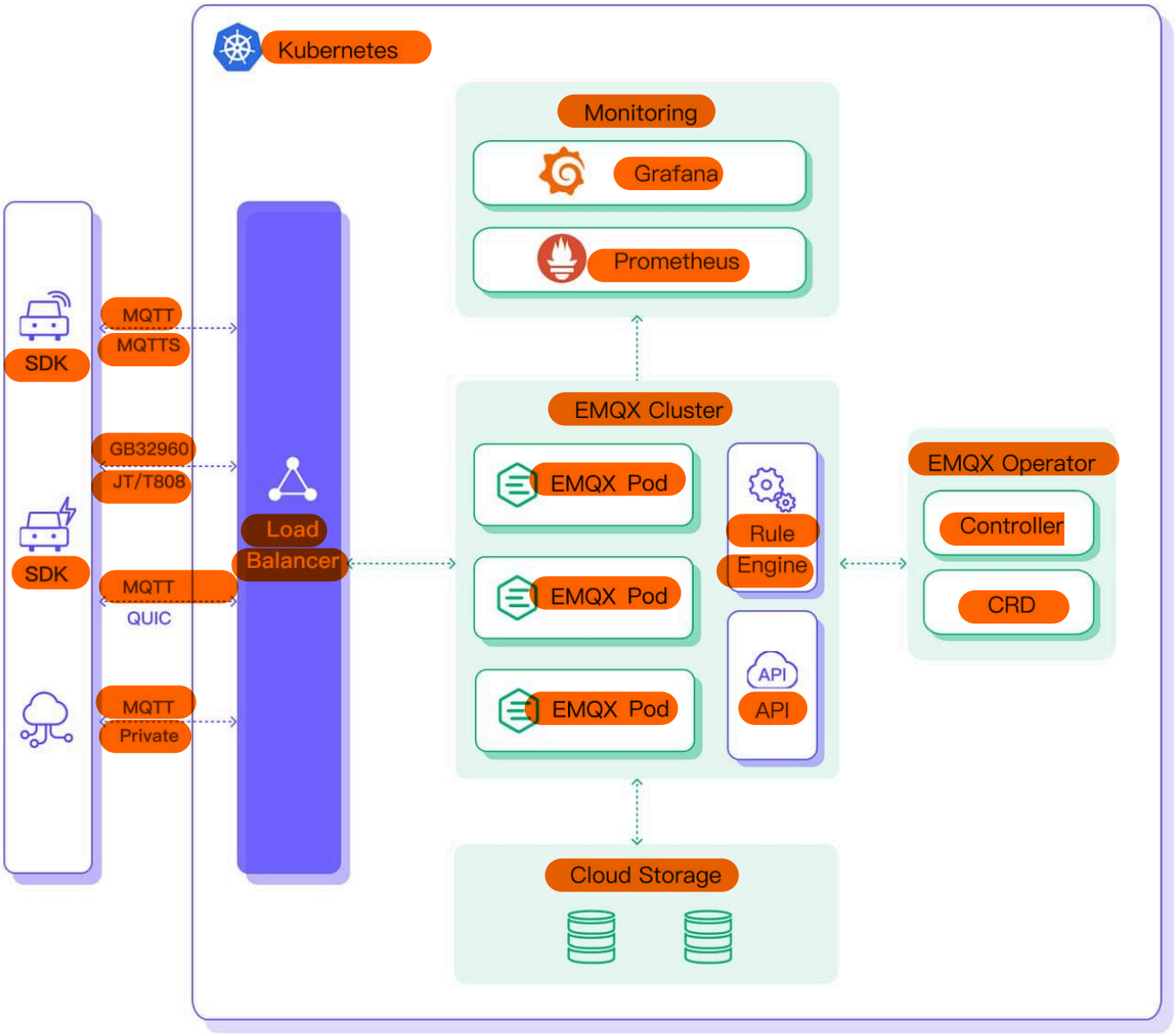
Scale MQTT horizontally by clustering brokers, distribute client connections across brokers, and use a shared/distributed state and message-routing layer so that publishers and subscribers can communicate regardless of which broker they connect to.

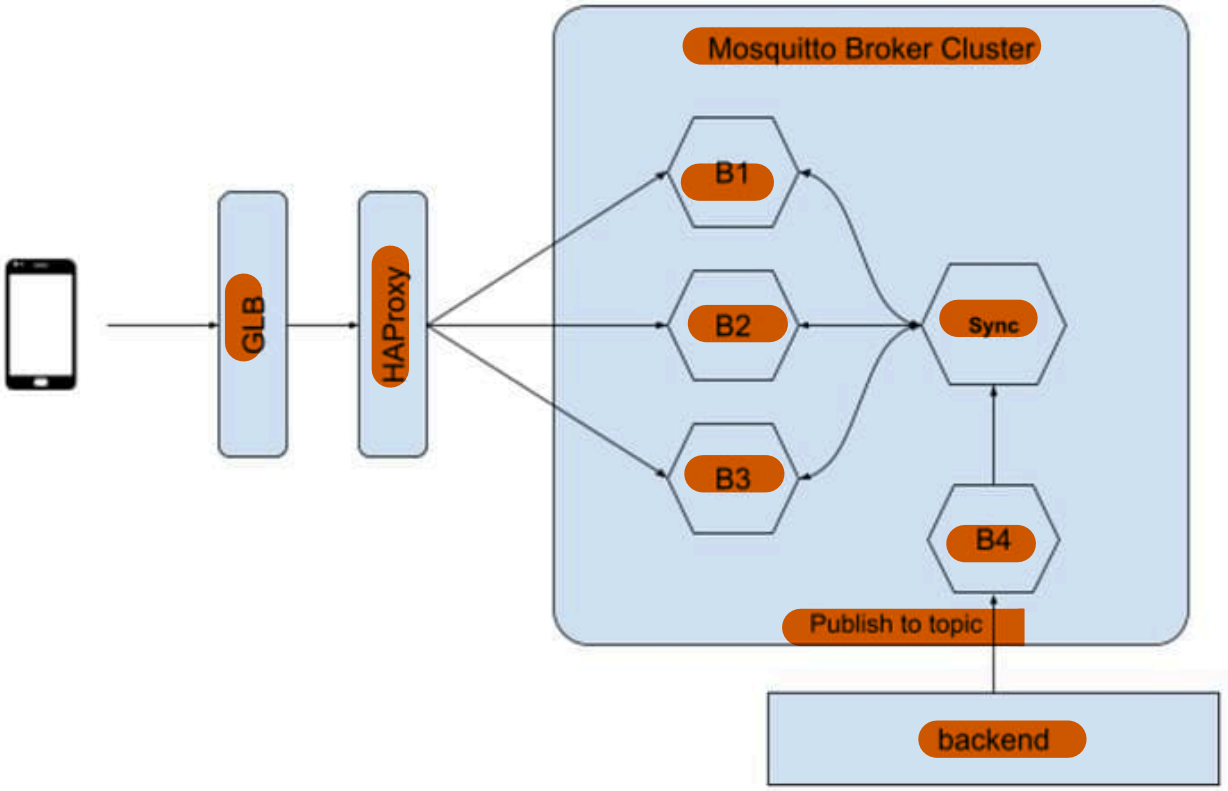
---

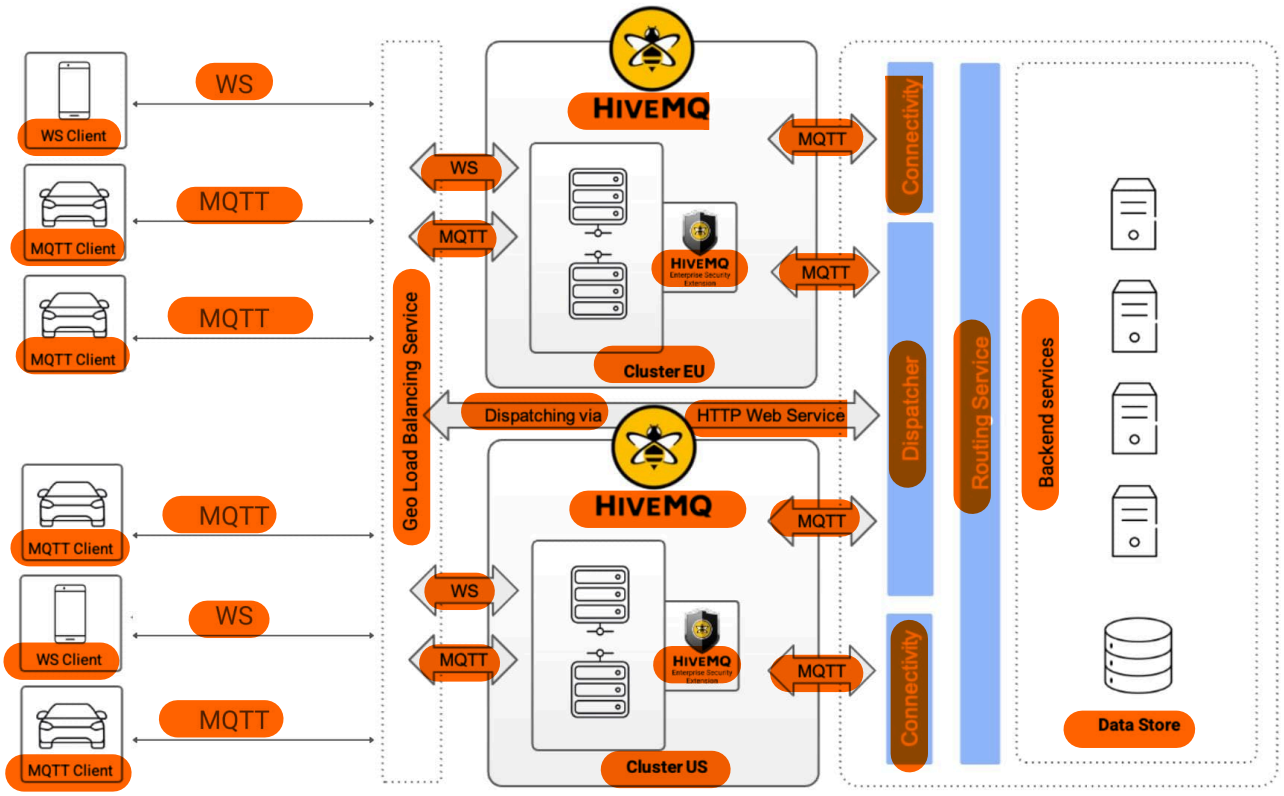
1. First understand the MQTT architecture

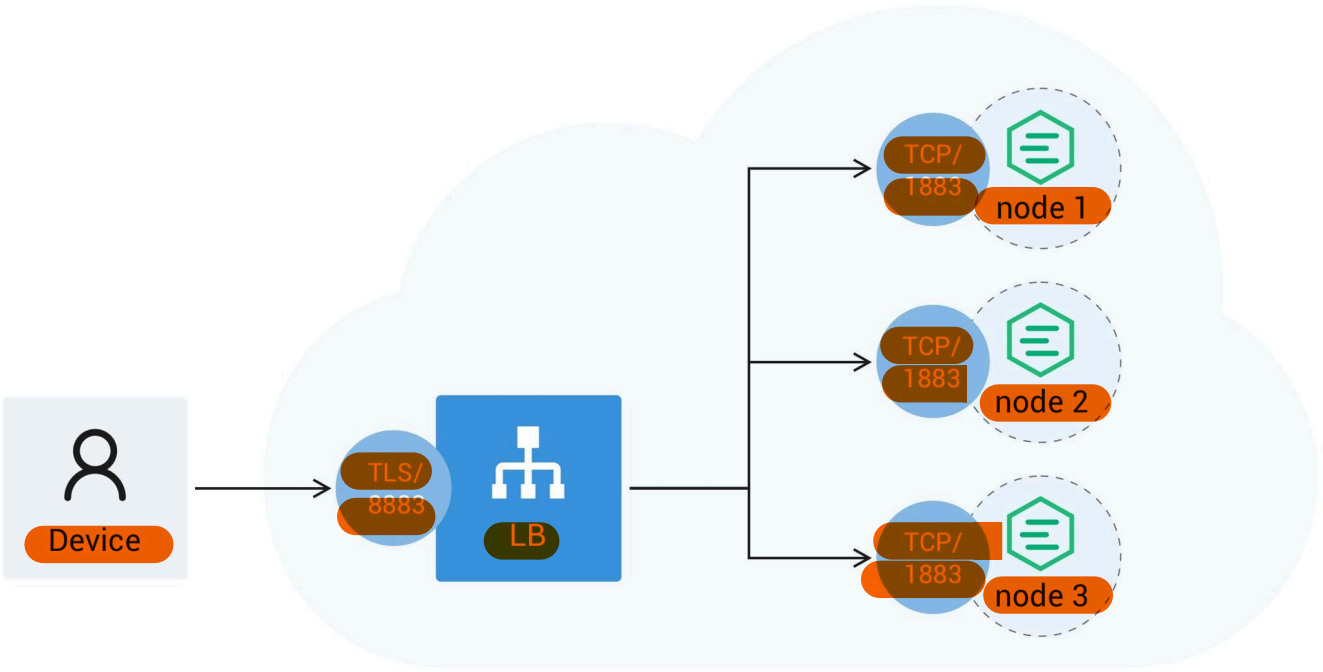



HiveMQ Cluster
TCP / MQTT
MQTT client
Broker node a
TCP / MQTT
MQTT client
Broker node b
Broker node c
TCP / MQTT
MQTT client

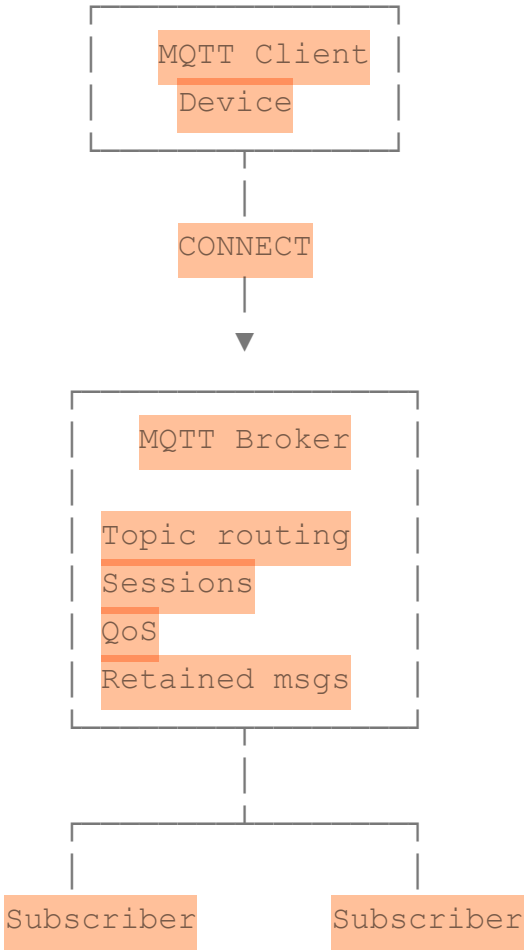

Kubernetes
Monitoring
Grafana
Prometheus
SDK
MQTT
MQTTS
GB32960
JT/T808
SDK
MQTT
QUIC
MQTT
Private
Load
Balancer
EMQX Cluster
EMQX Pod
EMQX Pod
EMQX Pod
Rule
Engine
API
API
EMQX Operator
Controller
CRD
Cloud Storage


Mosquitto Broker Cluster
B1
B2
B3
B4
Sync
GLB
HAProxy
Publish to topic
backend


WS
WS Client
MQTT
MQTT Client
MQTT
MQTT Client
MQTT
MQTT Client
WS
WS Client
MQTT
MQTT Client
Geo Load Balancing Service
HIVEMQ
WS
MQTT
HIVEMQ
Enterprise Security Extension
Cluster EU
Dispatching via
HTTP Web Service
HIVEMQ
WS
MQTT
HIVEMQ
Enterprise Security Extension
Cluster US
MQTT
MQTT
MQTT
MQTT
Connectivity
Dispatcher
Routing Service
Connectivity
Backend services
Data Store



At a basic level:



The broker is responsible for:

- TCP/TLS connections
- authentication
- authorization

- subscriptions
- topic matching
- message routing
- QoS handling
- retained messages
- persistent sessions
- keep-alive
- connection lifecycle

The broker therefore becomes the **primary scalability bottleneck**.

---

2. What exactly needs to scale?

When designing an MQTT platform, don't simply say:

> "We need to scale the broker."

Break scalability into dimensions.

Dimension	Example
Concurrent connections	10M devices
Connections/sec	100K CONNECT/s
Messages/sec	1M msg/s
Message size	1–100 KB
Subscriptions	50M
Topic count	Millions
Fan-out	1 → 1000 subscribers
QoS	0, 1, 2
Persistent sessions	Millions
Retained messages	Millions
Offline messages	Millions
TLS handshakes	Very high

Dimension	Example
Geographic distribution	Global

This distinction is extremely important in interviews.

A system supporting:

```
10 million connections
```

is not necessarily capable of:

```
10 million messages/sec
```

And a system supporting:

```
1M msg/sec
```

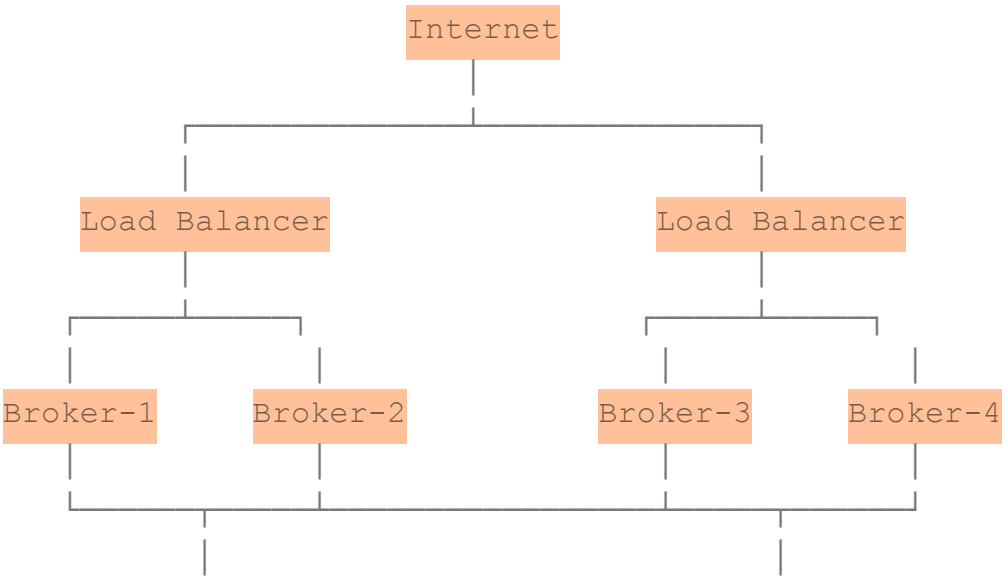
may fail if a single message needs:

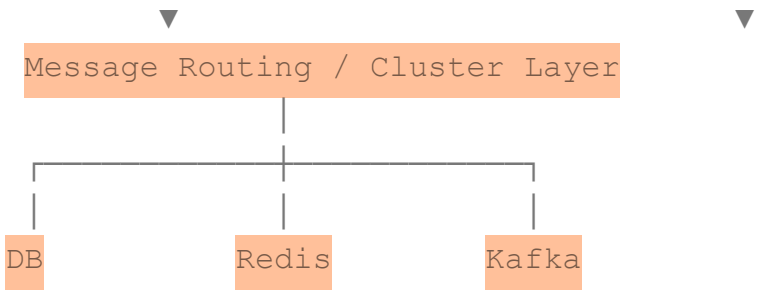
```
1 → 100,000 fan-out
```

---

3. The basic scalable architecture

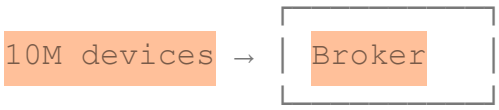
A production MQTT system generally looks like:



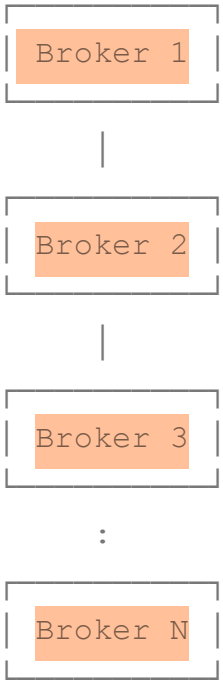


The key is **horizontal broker scaling**.

Instead of:



use:

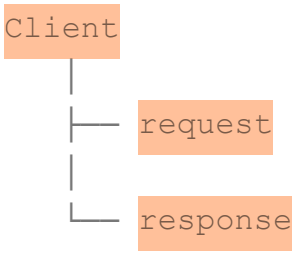


---

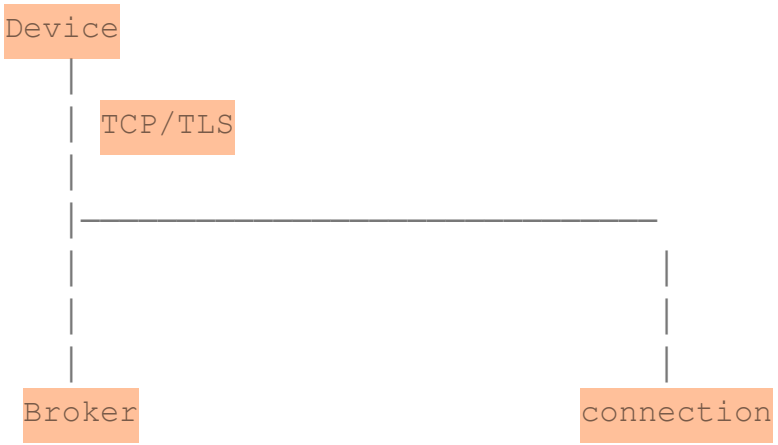
4. Challenge #1 — Long-lived TCP connections

This is one of the most important MQTT scaling issues.

HTTP services generally deal with short-lived requests:



MQTT clients maintain persistent connections:



Therefore:

```
10M devices
≈
10M persistent TCP connections
```

This creates pressure on:

- file descriptors
- memory
- TCP connection tables
- TLS state
- kernel networking
- event loops
- CPU
- connection tracking

---

5. Horizontal broker scaling

Suppose one broker can reliably handle:

```
500K connections
```

and we need:

```
10M connections
```

Then roughly:

```
10M / 500K = 20 brokers
```

In reality, provision additional capacity:

```
20 brokers
+
capacity buffer
+
failure capacity
```

For example:

```
30 brokers
```

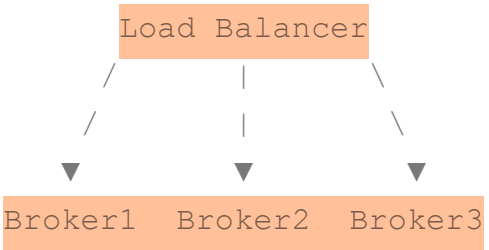
to tolerate failures and traffic spikes.

This is the same fundamental principle as scaling stateless application servers, except MQTT brokers have **connection and session state**.

---

6. Load balancing MQTT connections

A load balancer distributes MQTT clients:



However, MQTT connections are persistent.

Therefore the load balancer doesn't continuously rebalance an existing connection.

It only decides:

> **Which broker receives this new TCP connection?**

Once connected:

```
Device A → Broker 1
```

```
Device A stays on Broker 1
```

until it disconnects.

---

7. Sticky sessions

MQTT often benefits from connection affinity.

For example:

```
Device 123
    |
    ▼
Broker 7
```

All traffic remains:

```
Device 123
    ↓
Broker 7
```

This is naturally achieved because the TCP connection itself is persistent.

You generally don't need HTTP-style sticky-session cookies.

But you need to consider what happens after:

```
Broker 7 crashes
```

The device reconnects and may land on:

```
Broker 3
```

Therefore **session state cannot exist only in broker-local memory** if persistent sessions are required.

---

8. The hardest problem: cross-broker messaging

Consider:

```
Device A
   |
   ▼
Broker 1
```

and:

```
Device B
   |
   ▼
Broker 7
```

Both use:

```
factory/temperature
```

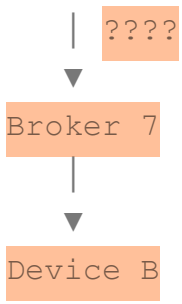
Device A publishes:

```
factory/temperature = 72
```

But Broker 1 doesn't directly own Device B's connection.

So:

```
Broker 1
   |
```

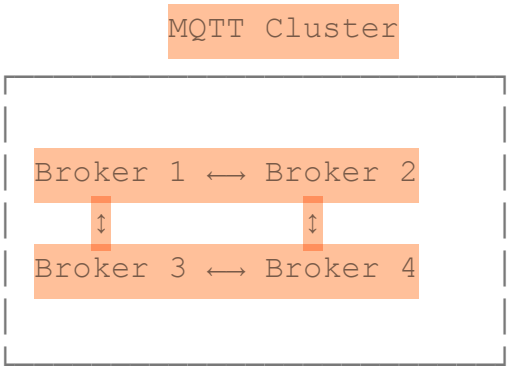


This is the fundamental distributed MQTT problem.

---

9. Broker clustering

A cluster creates a logical MQTT system:



If:

```
Client A → Broker 1
Client B → Broker 4
```

and:

```
A publishes topic X
```

the cluster routes the message toward Broker 4.

This requires some form of:

Subscription topology

```
Topic
  │
  ├── Broker 1 → Subscriber A
  ├── Broker 2 → Subscriber B
  └── Broker 4 → Subscriber C
```

Different MQTT broker implementations solve this differently.

Examples include clustered/distributed broker architectures such as:

- EMQX
- HiveMQ
- VerneMQ
- Mosquitto deployments with external clustering/routing components

The architectural principle matters more than the specific broker in an interview.

---

10. Don't broadcast every message to every broker

A naive implementation might do:

```
Broker 1 publishes Topic X

        ↓

Broker 2
Broker 3
Broker 4
Broker 5
...
Broker 100
```

This is disastrous at scale.

If:

```
B = number of brokers
```

```
M = messages/sec
```

then naive broadcast can approach:

```
O(M × B)
```

inter-broker traffic.

Instead, maintain routing information:

```
Topic
   ↓
Broker(s) containing subscribers
```

For example:

```
factory/1/temperature

        ↓

Broker 3
Broker 8
Broker 11
```

Only those brokers receive the publication.

---

11. Topic routing

A topic hierarchy is extremely useful:

```
company/
   factory/
      001/
         temperature
         pressure
         humidity
      002/
```

```
temperature
pressure
```

For example:

```
factory/001/temperature
```

Subscribers could use:

```
factory/001/#
```

or:

```
factory/+/temperature
```

The broker needs an efficient subscription matching structure.

This becomes increasingly important when you have:

```
100M subscriptions
```

---

12. Subscription index

Conceptually:

```
Subscription Trie

factory
   |
   ├── 001
   |     ├── temperature
   |     └── humidity
   |
   |
   └── 002
         └── temperature
```

A trie/tree-like structure can make topic matching more efficient than scanning every subscription.

For wildcard subscriptions:

```
factory/+/temperature
```

the broker must support:

```
+
#
```

efficiently.

At large scale, **topic design becomes a performance concern**, not merely an API design concern.

---

13. Separate connection state from durable state

This is a very important architecture principle.

You can divide MQTT state into:

Ephemeral connection state

```
Broker memory

- TCP connection
- socket
- event-loop state
- keepalive timer
- authentication context
```

Distributed/durable state

```
Shared infrastructure

- subscriptions
- persistent sessions
- QoS state
```

```
- offline messages
- retained messages
- device metadata
```

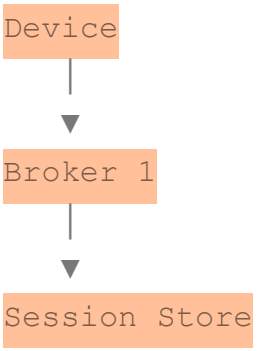
This allows:

```
Broker failure
      ↓
Device reconnect
      ↓
Another broker
      ↓
Restore session
```

---

14. Distributed session store

For persistent MQTT sessions:



Potential technologies:

```
Redis
Cassandra
DynamoDB
PostgreSQL
specialized broker storage
```

The correct choice depends on requirements.

For extremely large IoT workloads, a distributed KV/wide-column store is often more suitable than a single relational database.

---

15. Offline messages

Suppose:

```
Device B
```

disconnects.

Then:

```
Publisher
    |
    ▼
MQTT broker
    |
    ▼
Offline queue
```

When B reconnects:

```
Device B
    |
    ▼
Broker
    |
    ▼
Offline messages
```

This can become one of the largest storage requirements.

Therefore impose:

- maximum offline queue size
- message TTL
- maximum session expiry
- per-device quotas
- backpressure

Otherwise one device can consume enormous amounts of storage.

---

16. MQTT QoS is a scalability trade-off

MQTT has three QoS levels:

```
QoS 0 → At most once
QoS 1 → At least once
QoS 2 → Exactly once
```

From a scaling perspective:

```
QoS 0
  ↓
lowest overhead

QoS 1
  ↓
ACK + retransmission state

QoS 2
  ↓
multiple protocol steps + more state
```

Therefore:

> **Don't use QoS 2 by default for high-volume telemetry**.

For telemetry:

```
QoS 0
```

may be sufficient.

For important commands:

```
QoS 1
```

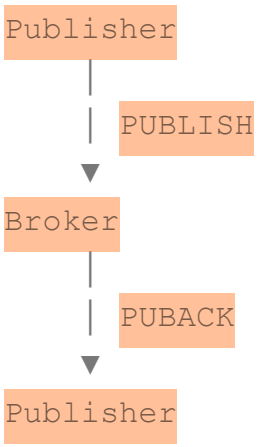
may be appropriate.

QoS 2 should be reserved for cases where its stronger delivery semantics are actually required.

---

17. MQTT QoS 1 scaling

With QoS 1:



The broker may need to maintain message state until the appropriate acknowledgment is received.

At:

```
1M messages/sec
```

this becomes significant.

You need to consider:

```
memory
+
persistent storage
+
network bandwidth
+
ACK processing
```

---

18. Retained messages

MQTT supports retained messages.

Example:

```
device/123/status
```

retains:

```
ONLINE
```

A new subscriber receives the latest retained value.

At scale:

```
millions of retained topics
```

means potentially millions of stored values.

Therefore retained-message storage should be treated as a separate capacity dimension.

---

19. Backpressure

Imagine:

```
Producer = 100K msg/sec

Consumer = 20K msg/sec
```

The queue grows:

```
100K
 ↓
80K backlog/sec
```

Eventually:

```
memory exhausted
```

Therefore implement:

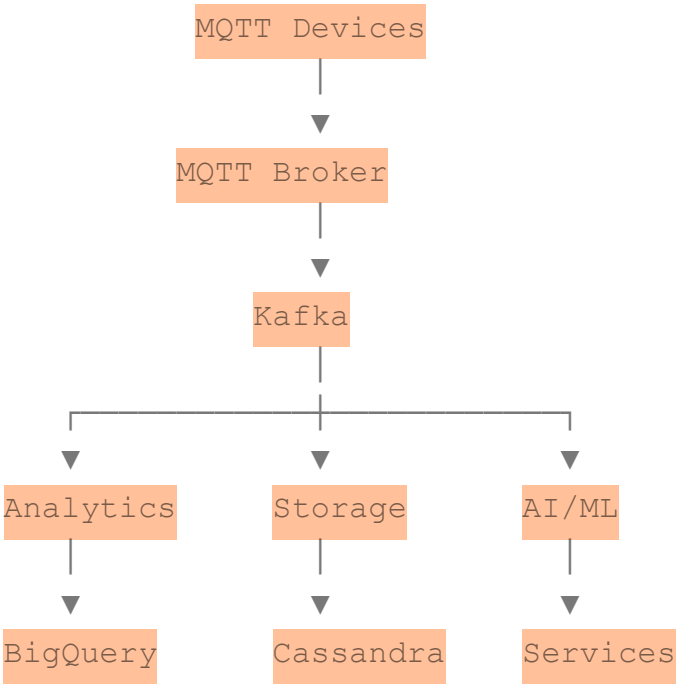
- bounded queues
- message TTL

- queue limits
- client quotas
- rate limiting
- connection throttling
- load shedding where acceptable

---

20. Kafka + MQTT

A very common architecture is:



This separates:

MQTT

Optimized for:

```
device connectivity
pub/sub
low latency
persistent connections
```

Kafka

Optimized for:

```
durable event streaming
replay
partitioning
consumer groups
large-scale processing
```

This is an excellent architectural separation.

---

21. Partitioning Kafka

Suppose MQTT publishes:

```
device/{deviceId}/telemetry
```

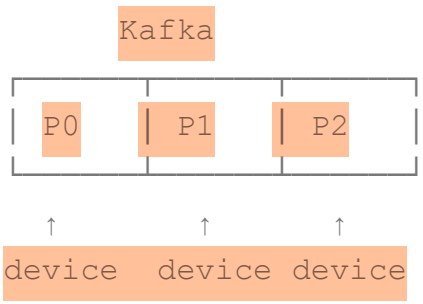
You can derive:

```
Kafka key = deviceId
```

Then:

```
hash(deviceId) → partition
```

Example:



This gives scalable ingestion and ordered processing **per partition/key**, rather than global ordering.

---

22. Don't put every MQTT operation into Kafka

This is an important design distinction.

Kafka can be excellent for:

```
telemetry
events
analytics
audit streams
data pipelines
```

But it doesn't automatically replace the broker's:

```
MQTT connection management
subscriptions
keepalive
QoS protocol handling
topic matching
```

So:

```
MQTT Broker
     +
Kafka
```

is generally more appropriate than:

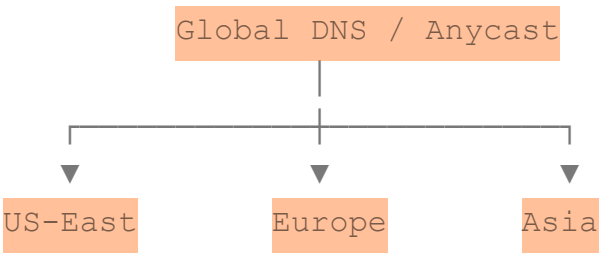
```
Kafka instead of MQTT
```

for MQTT device connectivity.

---

23. Multi-region architecture

For millions of devices, eventually you may need:



```
      |              |              |
MQTT Cluster  MQTT Cluster  MQTT Cluster
```

Devices should ideally connect to a nearby region.

Benefits:

- lower latency
- reduced WAN traffic
- better availability
- regional fault isolation

---

24. Avoid synchronous cross-region messaging

Suppose:

```
Device in India
      ↓
Asia MQTT Cluster
      ↓
Europe MQTT Cluster
```

Don't make every telemetry message synchronously wait for Europe.

Instead:

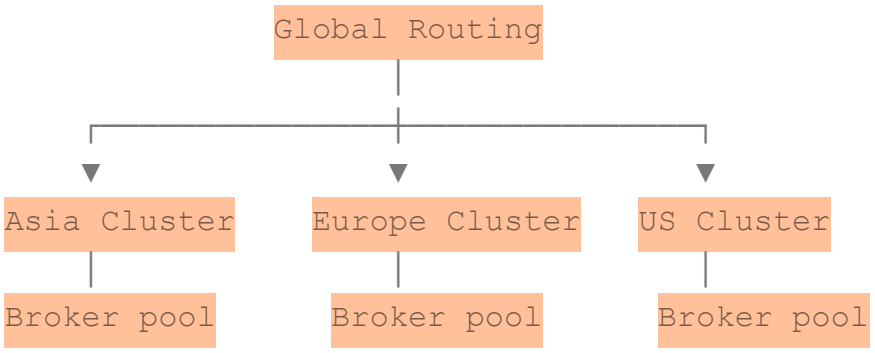
```
Asia
 │
 ├── local delivery
 │
 └── asynchronous replication
          ↓
       Europe
```

This reduces latency and isolates regional failures.

---

25. Global topic routing

A global architecture could look like:



Topics can be routed according to:

```
device ownership
tenant
geography
data classification
subscription location
```

---

26. Tenant-based sharding

For a SaaS IoT platform:

```
tenant-A
tenant-B
tenant-C
...
tenant-N
```

you can shard workloads:

```
Shard 1
  ├── Tenant A
  ├── Tenant D
  └── Tenant H
```

```
Shard 2
  ├── Tenant B
  ├── Tenant F
  └── Tenant K
```

This provides:

- isolation
- predictable capacity
- easier rate limiting
- easier migration
- noisy-neighbor protection

---

27. Device-based sharding

Another common strategy:

```
hash(deviceId) → broker cluster/shard
```

For example:

```
hash(deviceId) % N
```

This can distribute connection load evenly.

However, don't blindly use device hashing if your workload is highly skewed.

For example:

```
Tenant A = 5M devices
Tenant B = 10K devices
```

Tenant-based partitioning could create imbalance.

---

28. Hot topics

This is a subtle but important interview issue.

Suppose:

```
factory/all/temperature
```

has:

```
5 million subscribers
```

and receives:

```
10K messages/sec
```

A single logical topic becomes a massive fan-out problem.

Instead, partition the topic:

```
factory/001/temperature
factory/002/temperature
factory/003/temperature
...
```

and let subscribers consume only what they need.

---

29. Fan-out is often the real bottleneck

Suppose:

```
1 publisher
100,000 subscribers
```

and:

```
1 KB message
```

One message requires roughly:

```
100,000 × 1 KB
≈ 100 MB
```

of outbound payload.

At:

```
100 messages/sec
```

that's:

```
≈ 10 GB/sec
```

of outbound traffic.

So:

> **Scaling MQTT isn't just about messages/sec. You must calculate fan-out × message size × message rate**.

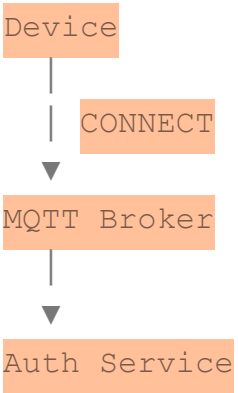
---

30. Authentication at scale

Millions of devices cannot necessarily perform expensive authentication operations on every message.

Authentication happens primarily during connection establishment.

Architecture:



Use:

- device certificates

- JWT
- API credentials
- OAuth-derived credentials where appropriate
- TLS client authentication

For massive IoT deployments, certificate-based device identity is common.

---

31. Authorization

Authentication answers:

> Who is this device?

Authorization answers:

> What topics may this device access?

For example:

```
device/123
```

may be allowed:

```
device/123/telemetry
device/123/status
```

but not:

```
device/999/commands
```

A scalable authorization model often uses topic patterns:

```
tenant/{tenantId}/device/{deviceId}/#
```

rather than storing an enormous ACL list per message.

---

32. TLS scalability

TLS can become a CPU bottleneck during massive connection churn.

For example:

```
100K devices reconnect simultaneously
```

can cause a huge TLS handshake spike.

Mitigations include:

- connection rate limiting
- session resumption where supported
- efficient TLS configuration
- hardware acceleration
- sufficient CPU capacity
- avoiding synchronized reconnects
- exponential backoff + jitter

The last point is particularly important.

If millions of devices reconnect simultaneously after an outage:

```
thundering herd
```

can bring the cluster down again.

---

33. Keep-alive scaling

MQTT clients send periodic keep-alive traffic.

Suppose:

```
10M devices
keepalive = 60 seconds
```

Then roughly:

```
10M / 60
≈ 166K keepalive events/sec
```

This isn't necessarily huge in bandwidth, but it creates:

- timer management
- packet processing
- event-loop work
- connection liveness checks

So keep-alive configuration matters.

---

34. Connection storm protection

Imagine a cloud outage causes:

```
10M devices
      ↓
all reconnect
```

You should implement:

```
Exponential backoff
        +
Random jitter
        +
Connection rate limiting
        +
Load shedding
```

For example:

```
1 sec
2 sec
4 sec
8 sec
16 sec
...
```

with randomized jitter.

---

35. Observability

At scale, MQTT requires specialized metrics.

Connection metrics

```
active_connections
connections/sec
disconnects/sec
TLS_handshakes/sec
connection_errors
```

Messaging

```
messages/sec
bytes/sec
publish_latency
delivery_latency
QoS1_ack_latency
QoS2_latency
```

Broker

```
CPU
memory
GC
event-loop utilization
file descriptors
network I/O
```

Queue

```
queue_depth
offline_messages
retained_messages
message_age
```

Cluster

```
inter-broker traffic
routing-table size
replication lag
cluster membership
partition health
```

---

36. Failure handling

Consider:

```
Broker 5 crashes
```

Connected devices detect the connection failure.

They reconnect:

```
Device
  |
  ▼
Load Balancer
  |
  ▼
Broker 8
```

If persistent session state is distributed:

```
Broker 8
  |
  ▼
Session Store
```

the device can recover its session/subscriptions according to MQTT version and session configuration.

---

37. Broker failure vs message durability

Don't confuse:

```
connection availability
```

with:

```
message durability
```

You might have:

```
99.99% broker availability
```

but still lose QoS 0 messages during failure.

That's expected because QoS 0 provides weaker delivery guarantees.

For critical messages, use appropriate QoS and durable infrastructure.

---

38. Capacity planning

For an interview, explicitly formulate the workload.

Example:

```
Devices              = 10M
Average msg/device   = 1 msg / 10 sec
Message size         = 1 KB
```

Messages/sec:

```
10M / 10
= 1M msg/sec
```

Ingress bandwidth:

```
1M × 1 KB
≈ 1 GB/sec
```

or roughly:

```
8 Gbps
```

before protocol/network overhead.

Now suppose average fan-out = 3:

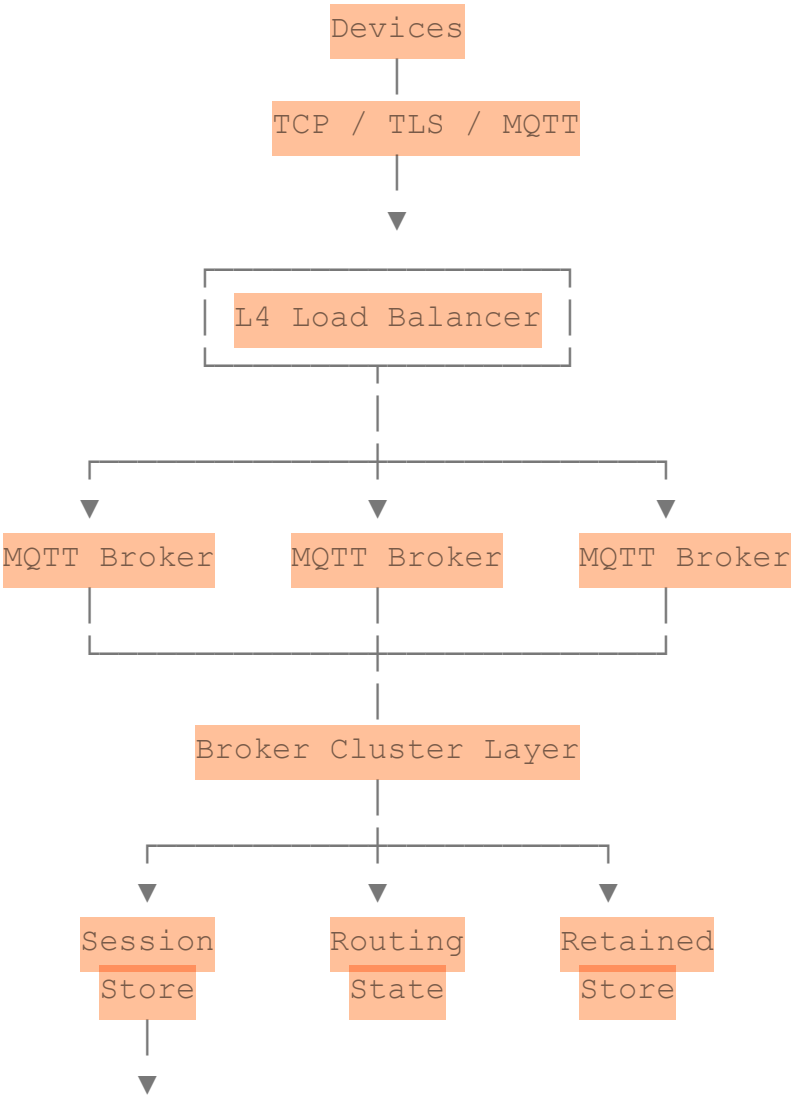
```
Outbound
≈ 3 GB/sec
```

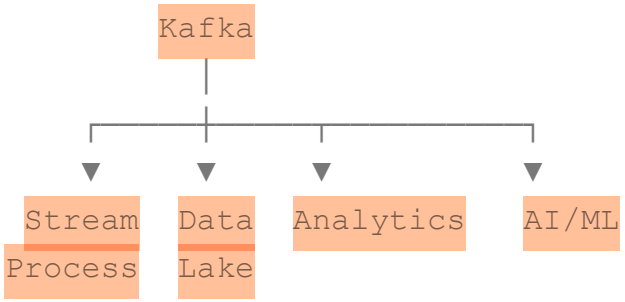
This immediately tells you that **network capacity and fan-out**, not merely CPU, may dominate.

---

39. A practical scalable architecture

For a large IoT platform:





---

40. What should go where?

A useful mental model:

Component	Responsibility
Load Balancer	Distribute new connections
MQTT Broker	Connections + MQTT protocol
Broker Cluster	Cross-broker routing
Session Store	Persistent session state
Distributed KV/DB	Metadata / retained state
Kafka	Durable event streaming
Stream Processor	Real-time processing
Object/Data Lake	Long-term telemetry
Redis	Caching / ephemeral coordination
API Gateway	REST/GraphQL management APIs
Device Registry	Device identity/metadata
Auth Service	Authentication
Policy Service	Authorization

---

41. MQTT vs HTTP scaling

This is a useful interview comparison.

	HTTP	MQTT
Connection	Usually request/response	Long-lived
Communication	Client → server	Pub/Sub
Broker	Not required	Required
Connection state	Lower	High
Network overhead	Higher	Very low
IoT suitability	Moderate	Excellent
Fan-out	Application responsibility	Native
Offline delivery	Application responsibility	Sessions/QoS
Scaling challenge	Request throughput	Connections + routing + fan-out
Typical LB	L7/L4	Usually L4 for MQTT/TCP

---

42. MQTT 3.1.1 vs MQTT 5 — scaling considerations

MQTT 5 adds features useful for large distributed systems, including:

- message expiry
- session expiry
- reason codes
- user properties
- flow control
- shared subscriptions
- improved control over delivery

Two particularly useful concepts for scalability are:

Message expiry

Don't let stale telemetry remain indefinitely:

```
message TTL = 60 seconds
```

Session expiry

Don't retain inactive device state forever:

```
session expiry = 24 hours
```

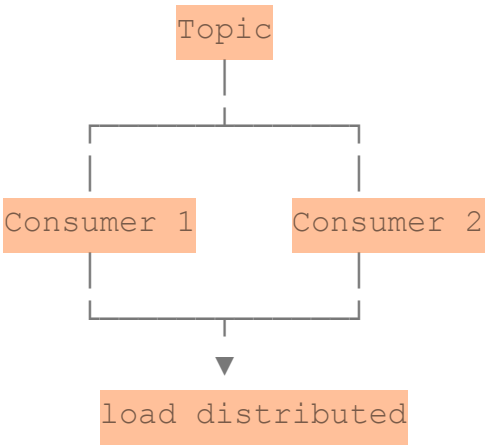
This prevents unbounded state accumulation.

---

43. Shared subscriptions

Shared subscriptions are extremely useful for scaling consumers.

Conceptually:



Instead of every consumer receiving every message, messages are distributed among members of a shared subscription group.

This is analogous to the consumer-group concept in Kafka.

It is especially useful for:

```
backend workers
command processors
telemetry processors
rule engines
```

---

44. Common mistakes in MQTT system design interviews

Mistake 1 — One giant broker

```
10M devices
    ↓
one broker
```

Bad failure domain and difficult capacity scaling.

Mistake 2 — Stateless MQTT brokers

MQTT brokers are not purely stateless because they maintain:

```
connections
subscriptions
QoS state
sessions
```

You need an explicit strategy for state.

Mistake 3 — Broadcast every message

```
Broker 1 → every broker
```

This creates unnecessary network traffic.

Mistake 4 — Store everything in Redis

Redis can be useful, but using it as the sole durable store for:

```
billions of telemetry messages
```

is usually inappropriate.

Mistake 5 — Use QoS 2 everywhere

This can introduce unnecessary protocol/state overhead.

Mistake 6 — Ignore reconnect storms

A system may handle:

```
10M connected devices
```

but fail when:

```
10M devices reconnect simultaneously
```

Mistake 7 — Ignore fan-out

```
1M msg/sec
```

sounds impressive until:

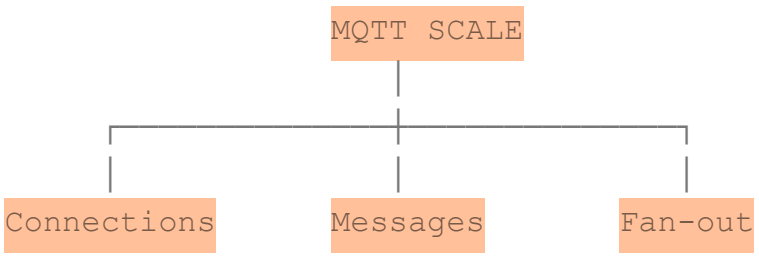
```
1M × 100 subscribers
```

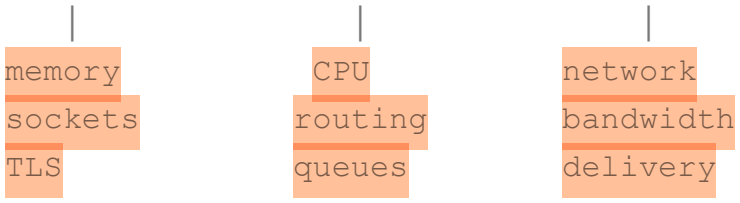
is considered.

---

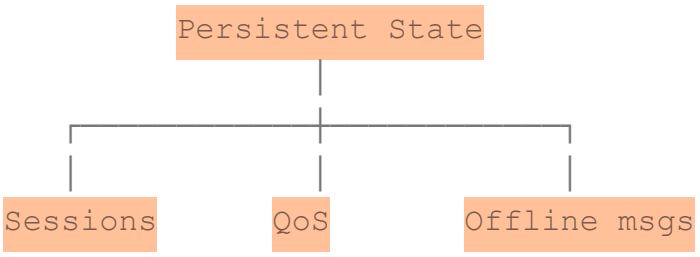
45. The scaling formula you should remember

For MQTT, think about **four independent dimensions**:





And add:



---

46. Interview-ready design

If asked:

> **"Design an MQTT system supporting 10 million IoT devices."**

I would structure the answer like this:

Step 1 — Requirements

```
10M concurrent devices
1M msg/sec
1 KB average message
QoS 0/1
99.99% availability
low latency
multi-region
```

Step 2 — Connection layer

```
Global DNS
    ↓
Regional L4 Load Balancer
    ↓
MQTT Broker Cluster
```

Step 3 — Horizontal scaling

```
N broker nodes
```

Scale based on:

```
connections/node
messages/sec/node
network/node
CPU/node
```

Step 4 — Cluster routing

```
Broker A
   ↓
subscription/routing metadata
   ↓
Broker B
```

Only route messages to brokers that actually have matching subscribers.

Step 5 — Persistent state

```
Distributed KV / DB
```

for:

```
sessions
subscriptions
retained messages
QoS state
```

Step 6 — Streaming

```
MQTT
 ↓
Kafka
```

```
  ↓
stream processing
```

for telemetry analytics and downstream systems.

Step 7 — Reliability

```
multi-AZ
multi-region
replication
reconnect backoff
jitter
rate limiting
```

Step 8 — Observability

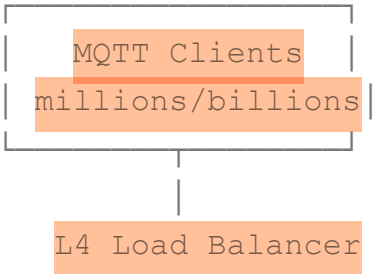
Monitor:

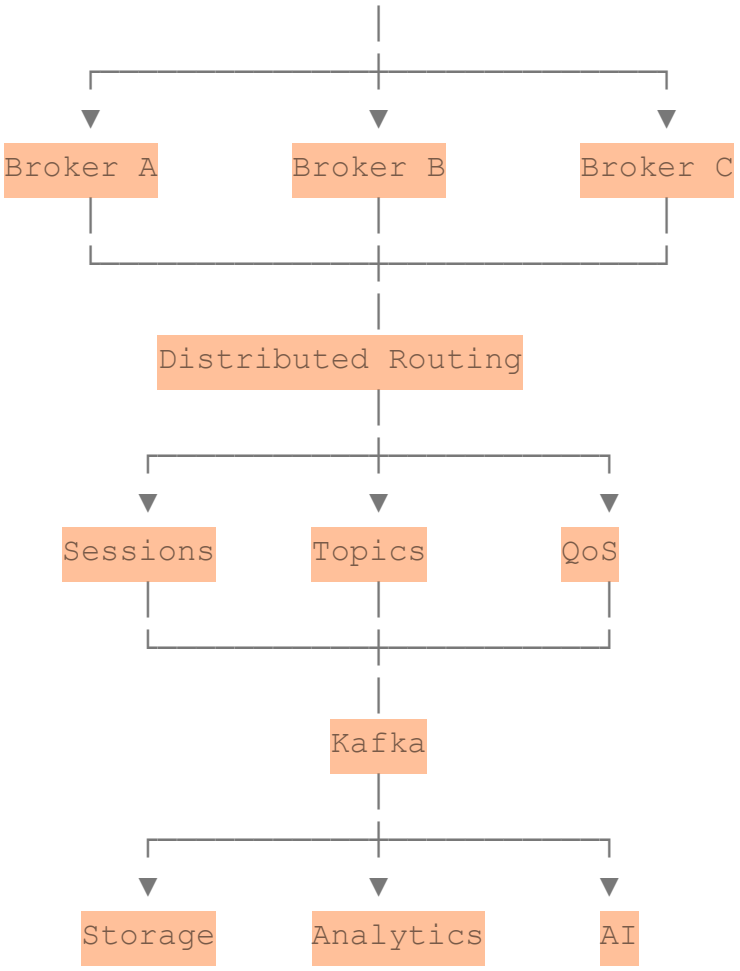
```
connections
msg/sec
latency
fan-out
queue depth
CPU
memory
network
cluster health
```

---

47. The most important mental model

For your system-design preparation, I'd remember MQTT scaling as:





The **four biggest scaling problems** are:

1. **Millions of persistent connections**
2. **Efficient cross-broker topic routing**
3. **Massive fan-out**
4. **Persistent MQTT state and offline delivery**

And the most important architectural principle is:

> **Keep the connection handling close to the MQTT broker, distribute brokers horizontally, distribute durable state independently, and use an event-streaming platform such as Kafka for downstream high-volume processing rather than forcing the MQTT layer to become your entire data platform.**

This is a particularly useful pattern to connect with the **WebSocket, RabbitMQ, and Cassandra scaling designs** you've been studying: MQTT combines the **long-lived connection problem of WebSockets** with the **broker/routing problem of messaging systems**, while Kafka can provide the durable event-stream backbone behind it.